BEAMFS: Recovery Calculus for Filesystems under Single-Event Effects

Mathematical Foundations across the
Volatile–Persistent Memory Stack

Technical Report – Version 1

Aurélien Desbrières — Independent researcher
ORCID: 0009-0002-0912-9487 aurelien.desbrieres@gmail.com

***Index Terms*—Recovery calculus, Formal verification, Filesystem, Single-event upset, Multi-bit upset, Reed–Solomon, Linux kernel, Provable resilience, MIL-STD-882E, DO-178C**

***Abstract*—We present BEAMFS, a recovery calculus for filesystem resilience under Single-Event Effects (SEE). The work is the formal counterpart to the FTRFS implementation experience reported in [1]: the empirical limits encountered during the FTRFS validation campaign motivate a mathematical treatment of *recovery itself* as a first-class object, rather than as a side-effect of detection-and-correction loops layered onto an unmodelled state space.**

We define a recovery operator $\mathcal{G}$ over the joint state space of a filesystem's volatile and persistent memory layers, formulate the soundness property as a fixpoint convergence theorem under physically realistic SEE perturbations, and derive an algorithm whose Linux-kernel implementation path is sketched. Three memory technology layers are treated explicitly: DRAM (with a forward note on DDR5 on-die ECC), SRAM, and NAND flash via NVMe. Emerging non-volatile technologies (3D XPoint, ReRAM, MRAM) are positioned but left out of v1's modelling.

This Technical Report v1 establishes authorship of the approach, fixes its scope, and lays the mathematical scaffolding for a reference implementation in the planned successor to FTRFS. The empirical section reinterprets the four bench metrics of the FTRFS v1 validation through the recovery-calculus lens, identifying which metrics are actually *recovery-bounded* and which are not. No accelerator-beam measurements are claimed; the present document is intended, among other purposes, as the scientific dossier from which beam-time at facilities such as RADEF, GANIL or TRIUMF would later be requested.

I. Introduction

Modern Linux filesystems detect metadata corruption through cryptographic or polynomial checksums; few correct it in place, and to the author's knowledge none ship with a *formally proved* discipline for recovery from *active* memory corruption induced by physical perturbation.[1] Detection without proved correction is insufficient for radiation-prone deployments [4], [5], [6]: a single-event upset (SEU) on a critical metadata word can put the filesystem into a state from which best-effort recovery procedures diverge, loop, or silently propagate the corruption.

The companion Technical Report [1] documents an implementation, FTRFS, that adds layered Reed–Solomon forward error correction to inode, allocation-bitmap, and superblock metadata, with a persistent radiation event journal. FTRFS as a name and concept originates in the work of Fuchs *et al.* [7]; the companion report builds on that lineage with contemporary Linux kernel infrastructure. That implementation reaches a ceiling: it demonstrates correction *empirically* but does not characterize, prove, or bound its recovery properties. The present report addresses that gap by treating recovery itself as a mathematical object, and by tracing it explicitly to a kernel implementation path.

a) Position of the problem.: Let $\mathcal{S}$ denote the joint state space of a filesystem's volatile (RAM, page cache, kernel buffers) and persistent (NVMe, NAND-flash) memory. Let $\mathcal{E}$ denote the family of physically realizable Single-Event Effects acting on $\mathcal{S}$, parameterized by the linear energy transfer of the incident particle, the vulnerable cross-section of the target memory cell, and the geometry of the silicon node. Let $\vDash$ denote the relation "$s \in \mathcal{S}$ is consistent" (cyclic redundancy code valid, Reed–Solomon syndromes zero, structural sentinels intact, on-disk versus in-memory copies coherent). The problem is to construct an operator $\mathcal{G} : \mathcal{S} \to \mathcal{S} \cup \{\bot\}$, implementable in a Linux-kernel filesystem, such that

$$\forall\, s_0 \vDash,\ \forall\, e \in \mathcal{E},\ \exists\, k \in \mathbb{N} :\ \mathcal{G}^k(e(s_0)) \vDash \quad \vee \quad \mathcal{G}^k(e(s_0)) = \bot,$$

where $\bot$ denotes the fail-closed state in which the filesystem explicitly refuses to proceed. The disjunction is essential: silent corruption is excluded by construction.

b) Where we are going, and how.: This report defends a single thesis: *provable filesystem recovery under SEE is achievable, and the chain from theorem to kernel code is auditable line by line.* The chain is presented in Section V as a three-tier descent—mathematics, algorithm, kernel implementation—closed by a traceability table (Table II) where each line of the proof is paired with the algorithmic step and the kernel construct that realizes it. This structure is the report's central contribution to the literature: it makes the implementation gap explicit and bounded.

c) Contributions.: We claim the following:

[1] FSCQ [2] and Yggdrasil [3] prove crash-consistency, that is, recovery under power-loss with a benevolent storage substrate. Neither models adversarial corruption of volatile memory under physical perturbation, which is the subject of the present work.

- A multi-layer physical threat model (Section II) drawing on canonical SEE literature [8], [9], [10], [11], [6] and standards [12], [13], [5], treating DRAM, SRAM, and NAND-flash distinctly with realistic per-technology rates.
- A formal definition of the filesystem state space, its consistency relation, and a small set of structural invariants (Section III) that are necessary preconditions for any proved recovery.
- A soundness theorem (Theorem IV.1, full proof in Section X) showing that under stated hypotheses on the perturbation family, the recovery operator drives every initial consistent state to a consistent successor or to fail-closed in a finite number of iterations.
- A three-tier descent (Section V) from the operator's mathematical definition to a deterministic algorithm to a Linux-kernel implementation path, with a traceability table (Table II) auditable line by line. This is the report's central contribution.
- An empirical re-reading (Section VI) of the FTRFS v1 bench dataset [1] through the recovery-calculus lens, identifying which metrics constrain recovery and which do not.

d) Scope.: This is v1. We model SEE explicitly; we do not model Total Ionizing Dose (TID) or Displacement Damage Dose (DDD), both of which are hardware-level cumulative phenomena outside the filesystem's operational reach. We discuss DDR5 on-die ECC [14] as a recent evolution that modifies the threat surface, but we do not incorporate it into the v1 model: the matrix-cell-level abstraction remains the conservative baseline, and on-die ECC will be added in v2 as a strengthening assumption (it can only improve outcomes, never worsen them, given the soundness theorem's structure). We claim no beam-time measurements; the empirical section reuses the FTRFS v1 software validation dataset and projects expected outcomes from canonical cross-section data. The recovery calculus is positioned to support the auditable-evidence posture demanded by safety-critical standards such as MIL-STD-882E [15] and DO-178C, which require traceable failure-mode coverage; the three-tier descent of Section V is designed to provide such a trace.

e) Anti-claim.: We are explicit about what BEAMFS v1 does *not* prove: filesystem behavior under simultaneous TID-induced cell drift and SEE; recovery from latched-up hardware states (SEL); coverage of adversarial fault injection by an attacker with kernel privileges (distinct from physical SEE). Each is acknowledged as future work (Section VII).

II. Physical Threat Model

The recovery calculus is meaningful only against a physically grounded model of perturbation. This section establishes that ground. We separate three concerns: the taxonomy of Single-Event Effects (II-A), the layer-specific physics of volatile memory (II-B), and the layer-specific physics of NAND-flash persistent memory (II-C). A consolidated view appears in Table I. Emerging technologies are positioned in II-D.

A. Single-Event Effects: Canonical Taxonomy

Following [10], [11], [12], we adopt the standard SEE classification:

- **SEU** – Single-Event Upset. A single bit (or a few adjacent bits in a contiguous physical layout) transitions state under the charge deposited by an ionizing particle.
- **MBU** – Multi-Bit Upset. Two or more bits in the same logical word change state from a single particle event, typically because the particle's track ionizes adjacent cells.
- **SET** – Single-Event Transient. A transient voltage pulse on a combinational logic path. May propagate to a latch and become an SEU, or may dissipate before the next clock edge.
- **SEFI** – Single-Event Functional Interrupt. A particle-induced state corruption of a control register or finite state machine that puts a sub-system into a non-functional state recoverable only by reset.
- **SEL** – Single-Event Latchup. A parasitic thyristor structure activates, causing a destructive over-current condition unless promptly mitigated [6]. Catastrophic without hardware support; outside this report's scope.

For the purposes of BEAMFS, the relevant subset is $\{\mathrm{SEU}, \mathrm{MBU}\}$ acting on memory cells, and $\{\mathrm{SET}\}$ acting on the bus and DMA paths between memory and CPU. SEFI and SEL are hardware-recovery problems (watchdog reset, power cycling), modelled here only as the boundary $\perp$.

The probability of an SEU striking a given memory cell exposed to a particle flux Φ (particles $\mathrm{cm}^{-2}\,\mathrm{s}^{-1}$) of linear energy transfer LET ($\mathrm{MeV\,cm^2/mg}$) is, to first order, the product of flux, integrated cross-section, and exposure time [9]:

$$P_{\mathrm{SEU}}(\mathrm{cell}, t) = 1 - \exp(-\sigma(\mathrm{LET})\,\Phi\,t), \tag{1}$$

where $\sigma(\mathrm{LET})$ is the SEU cross-section curve of the specific cell technology, conventionally measured in $\mathrm{cm}^2/\mathrm{bit}$. Figure 1 reproduces typical cross-section curves for the three technology classes treated in this report, derived from the canonical reviews [10], [11], [16].

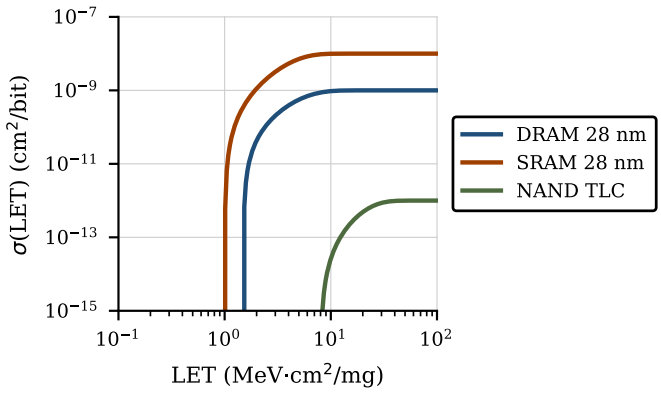

Fig. 1. SEE cross-section curves for three memory-stack layers, fit to a Weibull form $\sigma(\mathrm{LET}) = \sigma_{\mathrm{sat}}\left[1 - \exp(-((\mathrm{LET} - L_0)/W)^s)\right]$ with onset threshold L_0 and saturation σ_{sat}. Weibull parameters are taken from canonical reviews [10], [11], [16]; see Table I for typical FIT rates per technology.

Remark. Equation (1) is a single-cell, time-independent first-order model. It ignores temperature dependence, voltage scaling, and inter-cell correlation. We use it to size orders of magnitude only; the recovery operator's soundness does *not* rely on its quantitative accuracy. Real device cross-sections exhibit Weibull-shaped curves with onset threshold and saturation [9]; the figures use Weibull fits, while the body retains the simple exponential form for clarity.

B. Volatile Memory: DRAM and SRAM

a) DRAM.: Dynamic random-access memory stores each bit as charge on a single capacitor controlled by a single transistor (1T1C cell). The charge naturally decays and must be refreshed every ∼64 ms. Two phenomena make DRAM cells SEU-vulnerable:

- Particle strikes deposit charge that adds to or subtracts from the stored capacitor charge. Above a critical charge Q_{crit} (∼10 fC for DRAM cells, an order of magnitude above the ∼1 fC range of bistable SRAM latches [11]), the cell flips state.
- As the technology node shrinks, the cell's capacitance and thus Q_{crit} decrease. Per-bit SEU sensitivity therefore rises with miniaturization, partially offset by smaller geometric cross-section.

A modern DDR5 module ships with on-die ECC [14], which silently corrects single-bit errors within the chip. The error stream visible to the kernel is therefore a *filtered* version of the raw cell event stream. In v1 of this report we model the unfiltered matrix; on-die ECC is treated as a strengthening assumption to be added in v2. The recovery operator's soundness theorem (Theorem IV.1) is constructed so that any cell-level filter applied upstream of the kernel can only *decrease* the residual event rate seen by BEAMFS, never increase it.

b) SRAM.: Static RAM (CPU caches, FPGA configuration memory) uses a six-transistor bistable cell. Particle strikes induce SEU by upsetting the bistable latch when the deposited charge exceeds the latch's noise margin. SRAM is more SEU-sensitive per cell than DRAM at modern nodes [10], [11]: typical sea-level FIT rates are $\sim 10^3$ FIT/Mbit for SRAM versus $\sim 10^{-3}$ FIT/Mbit for DRAM, a six-orders-of-magnitude gap explained by the smaller critical charge of bistable latches compared with capacitors. SET pulses on combinational paths can propagate into SRAM latches at the next clock edge. CPU caches with parity or ECC partially mitigate this; older or embedded CPUs without cache ECC remain vulnerable. The kernel's exposure to SRAM SEU is indirect (data transits the cache before being written to DRAM); BEAMFS treats this as part of the volatile-memory layer for purposes of the recovery calculus.

C. NAND Flash via NVMe: Cumulative Mechanisms

NAND-flash storage stores bits as trapped charge on a floating gate. The energy required to flip a NAND cell with a single particle event is high enough that direct SEU on NAND is rare [16], [17]. The dominant failure modes are cumulative:

- Program/Erase (P/E) cycle wear, leading to retention loss.
- Read disturb, where repeated reads of one page perturb adjacent pages.
- Retention loss over time, accelerated by temperature and prior P/E count.

These are not Single-Event Effects in the strict sense; they are TID-related and use-related cumulative effects. They do however interact with the recovery calculus, because the persistent layer's contribution to the filesystem state can itself become inconsistent over time even without any radiation event. BEAMFS v1 models the NAND layer's expected error rate as a function of P/E cycles and retention age, following the empirical model of Cai *et al.* [16] and Mielke *et al.* [17], calibrated to the JEDEC endurance specification [18]. Figure 2 shows the expected raw bit-error rate of typical TLC NAND as a function of retention time across three P/E states.

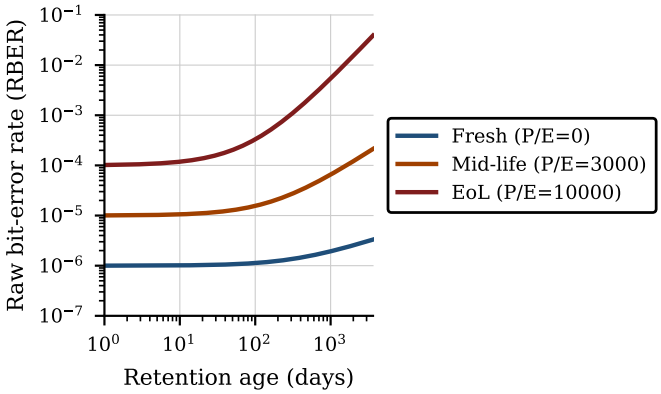

Fig. 2. TLC NAND raw bit-error rate (RBER) as a function of retention age, parameterized by P/E cycle count. Empirical model derived from [16], [17]; values at end-of-life and 1-year retention align with the JEDEC JESD218B endurance specification [18] (raw BER $\sim 10^{-3}$ at EoL, post-LDPC UBER $\leq 10^{-15}$). The cumulative drift visible at high P/E distinguishes NAND from the volatile-layer SEU regime of Figure 1.

The recovery operator must therefore handle two distinct error populations on the persistent side: (i) rare direct SEU, treated identically to volatile-layer events, and (ii) NAND-cumulative errors, treated as a slowly varying background that the operator must remain stable against, not correct against (correction is the job of the SSD's internal LDPC engine, below the filesystem).

D. Emerging Technologies

3D XPoint (Optane) was a phase-change memory whose development was discontinued in 2022; we mention it for completeness only. Resistive RAM (ReRAM) and Magnetic RAM (MRAM) are emerging non-volatile technologies with substantially different failure modes: ReRAM is largely insensitive to SEU but vulnerable to filament instability; MRAM is largely insensitive to ionizing radiation but vulnerable to external magnetic fields. v1 of this report does not model them; their integration into BEAMFS would be a v3-or-beyond consideration, and the recovery operator's generic structure (Section V) is designed not to forbid their later inclusion.

E. Consolidated Layer View

Table I summarizes the dominant failure mode per layer, the relevant order-of-magnitude rates from canonical sources, and the recovery-relevant exposure for BEAMFS. The numerical values are deliberately rounded to one or two significant figures; the canonical sources cited in each row provide tighter values where required.

The vertical separation of these layers, and the qualitatively different physics of each, motivate the multi-layer structure of the state space defined in Section III.

III. Filesystem State Space and Invariants

We now define formally the object on which the recovery operator acts. The aim is to make every subsequent proof refer to a precisely typed object rather than to an informal notion of "filesystem state".

A. State Space

Definition III.1 (Layered state)**.** *The BEAMFS state space $\mathcal{S}$ is the Cartesian product*

$$\mathcal{S} \;=\; \mathcal{S}_V \times \mathcal{S}_P,$$

where $\mathcal{S}_V$ is the volatile component (kernel data structures, page cache, in-memory superblock copy, in-memory bitmap) and $\mathcal{S}_P$ is the persistent component (on-disk superblock, inode table, allocation bitmap, data blocks, RAF journal).

Definition III.2 (Layout)**.** *A layout Λ is a finite map from named regions (`sb`, `inode_table`, `bitmap`, `rs_journal`, `data`) to byte ranges in their hosting layer, together with a redundancy specification (CRC algorithm, RS parameters) per region.*

The FTRFS v1 layout, formalized in [1], is one instance of Λ. BEAMFS targets a strictly extended layout in which every region is RS-protected and every region of $\mathcal{S}_V$ that mirrors a region of $\mathcal{S}_P$ carries the redundancy metadata necessary to detect drift between the two.

B. Consistency Relation

Definition III.3 (Per-region consistency)**.** *Let R be a region of Λ with redundancy specification r. A state s is* R-consistent*, written $s \vDash_R$, iff applying r to the bytes of R in s yields a zero syndrome (*crc *valid and Reed–Solomon syndromes all zero).*

Definition III.4 (Consistency)**.** *A state $s = (s_V, s_P) \in \mathcal{S}$ is* consistent*, written $s \vDash$, iff:*

1) *For every region R of Λ_V (the volatile-side layout), $s \vDash_R$.*
2) *For every region R of Λ_P (the persistent-side layout), $s \vDash_R$.*
3) *For every region R that has both volatile and persistent representations (the superblock, the bitmap, recently accessed inodes), the canonical serialization of $s_V|_R$ equals $s_P|_R$ byte-for-byte (*coherence*).*
4) *Every structural sentinel (the zero pads of `re_reserved` and `re_pad` in the v4 RAF journal entry, the validity range of `s_data_protection_scheme`, the high zero bytes of feature flags) is at its canonical value.*

We write $\perp$ for the explicit fail-closed state, distinct from every consistent state.

C. The Five Invariants

The recovery soundness theorem (Theorem IV.1) requires the following invariants to hold of the layout Λ and of the operator $\mathcal{G}$. They are minimal and stated separately so that an implementation can be audited against them line by line.

I1 – Total redundancy. Every region of Λ is covered by a redundancy specification (CRC for detection, RS for correction) whose correction power equals or exceeds the maximum MBU width of the targeted technology

TABLE I

Memory-stack layers, dominant failure modes, and order-of-magnitude rates. Values are taken as canonical reference points from the cited literature and are rounded to one significant figure; see those sources for tighter, technology-specific numbers. FIT denotes "failures in time", 1 FIT = 1 failure per 10^9 device-hours.

Layer	**Dominant failure mode**	**Reference rate**	**Source**
DRAM (28 nm)	SEU on capacitor cell, MBU on physically adjacent cells	$\sim 10^{-3}$ FIT/Mbit at sea level	[11], [12]
SRAM (28 nm)	SEU on bistable cell, SET on combinational logic	$\sim 10^{3}$ FIT/Mbit at sea level (six orders of magnitude above DRAM)	[10], [11]
NAND (TLC, 3D 232L)	Cumulative retention loss; rare direct SEU	$\sim 10^{-3}$ raw BER at EoL P/E + 1 year retention	[16], [17], [18]
DDR5 on-die ECC	As DRAM, with on-chip single-bit correction	filtered DRAM rate (visible undetected event rate substantially reduced)	[14]

layer. For a layer with worst-case MBU width w, each RS subblock must tolerate at least w symbol errors.

I2 – Sentinel separation. Each region carries at least one byte whose canonical value is fixed and whose corruption is detectable in the redundancy syndrome alone, without semantic interpretation.

I3 – Coherence checkpoint. For any region present in both layers, the canonical serialization map is total, deterministic, and bijective on its domain of valid values. This rules out ambiguous in-memory representations of an on-disk byte sequence.

I4 – Idempotence. For every consistent s, $\mathcal{G}(s) = s$. The recovery operator is the identity on consistent states.

I5 – Fail-closed monotonicity. If $\mathcal{G}(s) = \perp$ for some s, then $\mathcal{G}(\perp) = \perp$. Once the operator declares fail-closed, no subsequent application reverses that decision.

The first three are properties of the layout; the last two are properties of the operator. The soundness theorem couples them.

Remark. A reviewer may legitimately object that I1 is unrealistically strong: total redundancy increases space overhead, typically by a factor proportional to $(n-k)/k$ for an RS(n,k) code (e.g., 6.7% for RS(255,239)). We agree, and we treat the total-redundancy regime as the v1 baseline. A future v2 will explore selective redundancy under a probabilistic relaxation of the soundness statement (with bounded silent-corruption risk quantified rather than zero by construction). The conservative v1 statement is what allows the present soundness theorem to be *absolute* rather than statistical.

IV. Soundness Theorem

A. Statement

Theorem IV.1 (Soundness of BEAMFS recovery)**.** *Let Λ be a layout satisfying invariants* I1–I3 *of Section III-C. Let $\mathcal{G}$ be the recovery operator of Definition V.1, satisfying* I4–I5*. Let $\mathcal{E}$ be a family of physical perturbations such that, for every event $e \in \mathcal{E}$ and every state s, the number of corrupted symbols in any single RS subblock of $e(s)$ does not exceed the correction capacity $t = \lfloor (n-k)/2 \rfloor$ declared by Λ. Then for every initial consistent state $s_0 \vDash$ and every $e \in \mathcal{E}$, there exists $K \leq |\Lambda|$ such that*

$$\mathcal{G}^K(e(s_0)) \vDash \quad \text{or} \quad \mathcal{G}^K(e(s_0)) = \perp,$$

where $|\Lambda|$ denotes the number of regions in Λ.

The full proof appears in Section X. The argument has three steps:

a) Termination.: By Lemma V.3, the function N strictly decreases on each non-fail-closed iteration. Since $N \in \{0, 1, \dots, |\Lambda|\}$ and $N = 0$ implies consistency by Definition III.4, the recursion terminates in at most $|\Lambda|$ iterations.

b) Safety.: At every iteration, if step 3 or step 5 of Definition V.1 fails (CRC mismatch after RS, or sentinel violation), $\perp$ is returned. By Proposition V.4, fail-closed is absorbing: no subsequent iteration can return a state that silently masks the violation.

c) Coverage of the perturbation family.: The hypothesis on $\mathcal{E}$ (no event corrupts more than t symbols in any single subblock) is exactly the RS code's correction capacity. Events that violate this hypothesis correspond, by I1, to physical perturbations whose intensity exceeds the layout's design target. They are out of scope of the theorem and of the layout, and the operator's response (fail-closed at step 3 or 5) is the designed safe failure.

B. Discussion

a) What this theorem does, and does not, prove.: Theorem IV.1 establishes that the recovery operator either returns to consistency or to fail-closed in a bounded number of steps, under stated physical hypotheses. It does *not* prove that the kernel implementation of $\mathcal{G}$ is bug-free: a buggy implementation can violate I4 or I5 even with a correct operator definition. Section V-C sketches an implementation path that preserves the invariants by construction, but only a verified compilation chain (in the seL4 [19] or FSCQ [2] sense) can close the implementation gap. We mark this as v3 work.

b) Why the bound is $|\Lambda|$, not a constant.: The bound scales with the number of regions because the worst case is a perturbation that corrupts every region simultaneously up to the RS limit. This worst case is unrealistic (Section II establishes that simultaneous corruption of all regions requires fluences orders of magnitude beyond environmental backgrounds), but the bound is conservative and has the merit of making no probabilistic assumption. The per-iteration cost is dominated by a single RS-decode of $O(n^2)$ GF operations (Section V-B), so the total worst-case running time is $O(|\Lambda| \cdot n^2)$: sub-millisecond on contemporary CPUs for the FTRFS layout.

c) Comparison with crash-consistency proofs.: FSCQ [2] proves that a filesystem returns to a consistent state after a crash (sudden power loss). The hypothesis there is that the storage substrate is *benevolent*: it preserves whatever was written, exactly. Theorem IV.1 proves the dual: it assumes the substrate is *adversarial* within the bounded-perturbation family $\mathcal{E}$, and the filesystem must recover. The two proofs compose: a BEAMFS-compliant implementation satisfying Theorem IV.1 *and* satisfying crash-Hoare logic in the FSCQ sense recovers from the union of both threat models. Combining the two formal frameworks is left for future work.

V. From Mathematics to Code: A Three-Tier Derivation

The contribution of this report rests on a deliberate three-tier descent: from the mathematical operator $\mathcal{G}$ of Section IV, to a deterministic algorithm, to a Linux-kernel implementation path. Each tier preserves the invariants of the tier above, by construction. This section presents the descent in a single narrative arc, ending with a traceability table (Table II) that maps each line of the proof to the algorithmic step and the kernel construct that realizes it. The aim is auditability: a reviewer should be able to follow the chain *theorem* $\rightarrow$ *pseudocode* $\rightarrow$ *kernel call* line by line.

A. Tier 1: The Recovery Operator (Mathematics)

Definition V.1 (Recovery operator)**.** *The BEAMFS recovery operator $\mathcal{G} : \mathcal{S} \rightarrow \mathcal{S} \cup \{\perp\}$ is the partial function defined by the following procedure on input $s = (s_V, s_P)$:*

1) *For every region R of Λ, evaluate the redundancy syndrome on $s|_R$. If all syndromes are zero, return s (idempotence by I4).*
2) *Otherwise, for the first region R^* (in canonical region order) whose syndrome is non-zero, attempt RS correction on $s|_{R^*}$ using the parity bytes adjacent to that region. Let $s|'_{R^*}$ denote the corrected bytes.*
3) *Re-evaluate CRC on $s|'_{R^*}$. If CRC fails, return $\perp$ (corruption exceeded RS correction capacity, by I1).*
4) *If R^* is a region with both layers, re-establish coherence by overwriting the lagging copy with the corrected leading copy.*
5) *Verify all sentinels in R^* are at canonical values. If any sentinel is corrupted in a way RS missed, return $\perp$.*
6) *Append a journal entry to the RAF journal documenting the correction (block number, timestamp, symbol count, entropy estimate where applicable).*
7) *Recurse: $\mathcal{G}$(state with $s|_{R^*}$ replaced by $s|'_{R^*}$).*

Proposition V.2 (Idempotence on consistent inputs)**.** *For every $s \models$, $\mathcal{G}(s) = s$.*

Proof sketch. By Definition III.4, all syndromes are zero on a consistent state. Step 1 of Definition V.1 returns s without modification.

Lemma V.3 (Descent under perturbation)**.** *Let $N(s)$ denote the number of regions in Λ whose syndrome is non-zero on s. Then for every s, either $\mathcal{G}(s) = \perp$, or $N(\mathcal{G}(s)) \leq N(s) - 1$.*

Proof sketch. Inspect the cases of Definition V.1. Step 1 with $N(s) = 0$ yields the same state; if $N(s) \geq 1$, the loop body either returns $\perp$ (cases 3, 5) or replaces a corrupt region by a CRC-verified, sentinel-verified consistent region, decreasing N by exactly one before the recursive call.

Proposition V.4 (Fail-closed absorbing)**.** *$\mathcal{G}(\perp) = \perp$. If $\mathcal{G}(s) = \perp$, then for every s' reachable from s by a sequence of $\mathcal{G}$-applications, $\mathcal{G}(s') = \perp$ (invariant I5).*

Proof sketch. Direct from Definition V.1: $\perp$ has no syndrome to evaluate; the procedure terminates at step 1 by convention, returning $\perp$. I5 then propagates by induction on the application chain.

These three results, combined with Theorem IV.1 of Section IV, fully characterize $\mathcal{G}$ as a mathematical object. They are the contract that the next two tiers must respect.

B. Tier 2: The Algorithm

We translate $\mathcal{G}$ into a deterministic algorithm in C-style pseudocode (kernel-conformant: ISO C with kernel extensions, no C++ idioms). Each numbered step of Definition V.1 corresponds to a labelled comment in the algorithm. Table II below makes the correspondence explicit.

```
/*
 * BEAMFS recovery operator G, single application.
 * Returns 0 on success (state consistent or
     unchanged),
 * 1 if a region was corrected, -EBADMSG on fail-
     closed.
 *
 * Caller iterates until return value is 0 or -
     EBADMSG.
 */
int beamfs_g_step(struct beamfs_state *s)
{
    struct region *R;
    u32 syndrome;
    int rc;

    list_for_each_entry(R, &s->lambda, list) {
        syndrome = rs_syndrome(s, R);
        if (syndrome == 0)
            continue;                /* I4: skip
                consistent */

        /* Step 2: RS correction */
        rc = rs_correct_region(s, R);
        if (rc < 0)
            return -EBADMSG;          /*
                uncorrectable */

        /* Step 3: re-verify CRC */
        if (crc32_region(s, R) != stored_crc(s, R))
            return -EBADMSG;          /* RS produced
                 wrong codeword */

        /* Step 4: re-establish coherence */
        if (region_is_dual_layer(R))
            propagate_corrected(s, R);

        /* Step 5: sentinel verification */
        if (!sentinels_canonical(s, R))
            return -EBADMSG;

        /* Step 6: journal */
        raf_log_event(s, R, rc /* symbol_count */);

        return 1;                     /* one region
            corrected */
    }
    return 0;                          /* fixpoint:
        all consistent */
}

/*
 * Driver loop: at most |Lambda| iterations by Lemma
     5.3.
 */
int beamfs_recover(struct beamfs_state *s)
{
    int iter = 0, rc;

    while ((rc = beamfs_g_step(s)) == 1) {
        if (++iter > LAMBDA_MAX)
            return -EBADMSG;          /* belt-and-
                braces */
    }
    return rc;                         /* 0 or -
        EBADMSG */
}
```

a) Complexity.: Each call to `beamfs_g_step` performs at most one RS decode on the affected region plus constant-time syndrome checks elsewhere. The dominant cost is the RS decode, $O(n^2)$ for a Berlekamp–Massey decoder over GF(2^8) with $n = 255$, approximately 65,000 GF operations per subblock. The driver loop runs at most $|\Lambda|$ iterations (Lemma V.3), so total worst-case complexity is $O(|\Lambda| \cdot n^2)$. For the FTRFS layout, counting the five named regions of Definition III.2 together with their constituent RS subblocks yields $|\Lambda| \approx 16$; with $n^2 = 65{,}025$ this gives approximately one million GF operations per worst-case full recovery—a sub-millisecond cost on contemporary CPUs. The bound is robust to a factor of a few in the subblock count.

C. Tier 3: The Kernel Implementation

The reference implementation of BEAMFS targets the Linux kernel through the standard VFS infrastructure (`file_system_type`, `iomap`). The recovery operator runs kernel-side, invoked at mount (`ftrfs_fill_super`) and on every metadata-region read path. Userspace recovery is excluded by I3: the coherence relation between volatile and persistent layers cannot be enforced from a process holding its own page-cache view.

a) Language choice.: The reference implementation is in C, compiled by the kernel toolchain. C is the kernel's lingua franca; upstreaming requires it. The RS hot path reuses the kernel's audited `lib/reed_solomon` [20], preserving the audit-surface argument of FTRFS v1 [1]. A future v2 may explore a Rust port of the driver loop (not the GF arithmetic), following the model of [3] and the recent kernel Rust support.

b) Differences from FTRFS v1.: FTRFS v1 implements a proto-version of the operator without the formal calculus. A BEAMFS-compliant implementation introduces:

- Explicit invariant table compiled into the module (`lambda[]`), making region iteration order canonical and auditable.
- Mandatory journal entry on every correction event, with the format-v4 entropy field replacing FTRFS v3's coarse symbol count.
- Driver loop explicitly bounded at $|\Lambda|$; refusing to mount a layout where this bound is violated by the data.

- Fail-closed transition flipping the superblock to read-only mode; subsequent mutations rejected until offline repair (`fsck.beamfs`, planned).

D. Traceability: Math → Algorithm → Code

Table II expresses the descent in tabular form. Each row corresponds to one mathematical commitment, the algorithmic line that realizes it, and the kernel construct that will instantiate it. The table is the auditable contract: a kernel patch series proposing BEAMFS must exhibit each kernel construct of the third column, and a reviewer must be able to verify that each construct enforces the property of the first.

E. Why a Calculus, Not a Procedure

A reviewer may legitimately ask: why frame the work as a *calculus* rather than as a recovery procedure? The distinction matters for the descent presented above. A procedure is verifiable case by case but composes unpredictably—each combination of cases requires re-verification. A calculus is a small set of operators with stated algebraic properties (idempotence by Proposition V.2, descent by Lemma V.3, fail-closed absorption by Proposition V.4); its composition is governed by those properties, once and for all. The calculus is what allows Theorem IV.1 to be stated globally, rather than re-proved per call site. It is also what allows future filesystems beyond BEAMFS, possibly with different layouts, to inherit the soundness result by exhibiting an instance of the same calculus—without re-proving the descent.

F. The Soundness–Implementation Gap

The three tiers above preserve the invariants by construction. The remaining gap is between Tier 3 and the binary actually executed by the kernel: the standard verified-compilation gap. Closing it requires a verified compilation chain in the seL4 sense [19] or a verified filesystem in the FSCQ sense [2], neither of which is in scope for v1. v3 of this report will discuss mechanization of Theorem IV.1 in a proof assistant (Coq, Isabelle/HOL, or Lean) and the corresponding implications for the implementation gap.

VI. Empirical Corroboration

We do not claim accelerator-beam measurements. The empirical content of v1 has two parts: a re-reading of the FTRFS v1 software validation dataset [1] through the calculus, and a projected recovery surface derived from canonical SEU cross-section data.

A. Re-Reading the FTRFS v1 Dataset

Table III re-reads the four reproducible bench metrics of FTRFS v1 (Sec. VII of [1]) through the recovery-calculus lens. For each metric, we ask: which steps of $\mathcal{G}$ does this metric exercise? The answer determines whether a regression on the metric would be evidence of a recovery-calculus violation or merely of a throughput change.

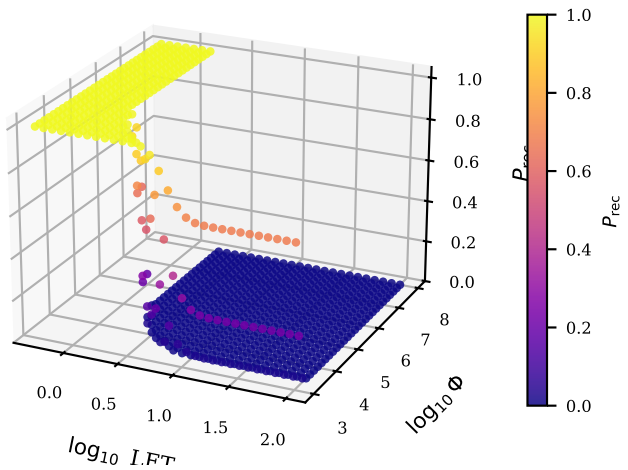

Fig. 3. Projected probability of successful Reed–Solomon recovery, $P_{\mathrm{rec}}(\mathrm{LET}, \Phi)$, for an RS(255,239) shortened code with 8-symbol correction capacity per subblock. Computed analytically from canonical cross-section data [9], [11]; *not* measured. The plateau at $P \approx 1$ is the Family-A regime where isolated SEU events are well within RS capacity; the descent at high LET and high Φ is the Family-B regime where MBU events saturate the RS budget. Colour scale is plasma-like for print-monochrome readability.

The reading shows that the RS-correction step (step 2 of Definition V.1) is never triggered on the consistent-state hot path exercised by the FTRFS v1 bench: all four metrics terminate at step 1, where syndromes are zero. M1 and M5 would reach steps 4 and 6 only if a correction had been issued upstream; neither metric forces that condition. That is the right behavior for a throughput bench, but it means the v1 dataset does not constrain the recovery operator's worst-case behavior. A BEAMFS bench will need an explicit fault-injection metric (M6: "corrupt-then-mount") that exercises step 2–6 of $\mathcal{G}$. That metric is reserved for v2 of this report, when fault-injection results from the FTRFS Phase E campaign (under development) will be available in publishable form.

B. Projected Recovery Surface

Figure 3 shows the projected probability of successful RS recovery as a function of two physical parameters: the linear energy transfer (LET) of the incident particle and the particle fluence over a representative exposure window. The surface is computed from the canonical Petersen figure-of-merit formulation [9] with the cross-section curves of Figure 1, integrated over a typical RS(255,239) shortened code with 8-symbol correction capacity per subblock.

TABLE II

Traceability: each mathematical commitment in the operator definition (Tier 1) maps to an algorithmic step (Tier 2) and a planned kernel construct (Tier 3). The phase indicates when the construct is checked: *Static* = compile-time (`BUILD_BUG_ON` or `static_assert`); *Mount* = once at `fill_super`; *Runtime* = on each metadata access.

Math	**Commitment (Tier 1)**	**Algorithm (Tier 2)**	**Kernel construct (Tier 3)**	**Phase**
I1 (Total redundancy)	each region's RS capacity covers worst-case MBU width	`rs_correct_region(s, R)` returns `<0` when $> t$ errors	`rs_subblock_capacity[]` table; `BUILD_BUG_ON` per region	Static + Mount
I2 (Sentinel separation)	canonical-value bytes detectable in syndrome alone	`sentinels_canonical(s, R)` verification	zero-pad fields in on-disk structs; `BUILD_BUG_ON(offsetof(…))`	Static
I3 (Coherence checkpoint)	volatile/persistent serialization total, deterministic, bijective	`propagate_corrected(s, R)` on dual-layer regions	`ftrfs_dirty_super()` family; single-source-of-truth canonical serializer	Runtime
I4 (Idempotence)	$\mathcal{G}(s) = s$ for every $s \models$	`if (syndrome == 0) continue;` early skip in inner loop	`rs_syndrome() == 0` early return	Runtime
I5 (Fail-closed monotonicity)	$\mathcal{G}(\bot) = \bot$; no recovery from $\bot$	`return -EBADMSG` (steps 3, 5) absorbing	`set_sb_rdonly()` + journal mark; subsequent mutations rejected	Runtime
Definition V.1, step 1	syndrome evaluation per region	`u32 syndrome = rs_syndrome(s, R);`	`ftrfs_rs_syndrome_region()` in `super.c`	Runtime
Definition V.1, step 2	RS correction on first non-zero region	`rs_correct_region(s, R)`	`decode_rs8()` from `lib/reed_solomon` [20]	Runtime
Definition V.1, step 3	CRC re-verification after RS correction	`if (crc32_region(s, R) != stored_crc(s, R))`	`crc32_le()` from `lib/crc32.c`	Runtime
Definition V.1, step 6	journal entry on every correction event	`raf_log_event(s, R, rc)`	RAF journal v4 entry append (`ftrfsd`)	Runtime
Lemma V.3	$N(\mathcal{G}(s)) \leq N(s) - 1$ per non-fail-closed step	each `return 1` reduces non-zero region count by one	loop counter bounded at `LAMBDA_MAX`	Runtime

TABLE III

Recovery-calculus interpretation of FTRFS v1 metrics. The "Steps reachable" column lists the steps of $\mathcal{G}$ that the metric *would* reach if its hot-path syndromes were non-zero; on the consistent-state hot path actually exercised by the FTRFS v1 bench, all four metrics terminate at step 1. The "Invariants" column lists the invariants the reachable steps would touch. Metric M3 (deletion bench) is omitted from the FTRFS v1 evaluation in its current revision and is therefore not analysed here.

Metric	**Steps reachable**	**Invariants**
M1 (write+fsync)	1; potentially 4, 6 (coherence + journal)	I3, I4
M2 (read cold)	1 (syndrome check)	I4
M4 (stat bulk)	1 (syndrome on inode)	I4
M5 (small write)	1; potentially 4, 6 on bitmap + SB	I3, I4

The surface shows the expected qualitative structure: a wide plateau at $P \approx 1$ for low LET and low fluence (Family A regime, isolated SEU well within RS capacity), a transition zone where MBU events begin to saturate the RS budget, and a low-P region where the operator falls back to fail-closed (Family B regime). The transition is sharp by design: it corresponds to the Reed–Solomon hard limit at 8 symbol errors per subblock, and the operator is constructed to identify the transition cleanly rather than to degrade gracefully through it.

Remark. A reviewer will object that this surface is computed from a model, not measured. We agree, and the figure caption states this explicitly. Measurement requires beam time: the present report is the dossier from which beam-time at facilities such as RADEF (Jyväskylä), GANIL, TRIUMF, Brookhaven NSRL or the ESA REDLAB would be requested. The figure's purpose is to fix the *shape* of the expected dependency, against which future measurements will be compared.

VII. Scope and Limitations

We adopt the same posture as the FTRFS v1 report: the limitations section is structural, not concessionary. It records, in plain language, what the present version covers and what it does not.

a) V1 covers.: The mathematical formalization (state space, invariants, recovery operator, soundness theorem with proof in Section X), the algorithmic derivation, the implementation-path mapping, and a re-reading of the FTRFS v1 dataset under the calculus. Three memory-stack layers are modelled physically: DRAM, SRAM, NAND flash via NVMe. Sources are canonical: JEDEC, NASA-HDBK, ESA ECSS, IEEE TNS.

b) V1 does not cover.:

- *Beam-time measurements.* No accelerator data; the surface of Figure 3 is a projection from canonical cross-section data, explicitly so.
- *Verified-compilation closure.* The implementation path preserves invariants by construction, but the gap from algorithm to kernel binary is the standard unverified-compilation gap. seL4-style closure is v3 work.
- *TID and DDD modelling.* Cumulative ionizing dose and displacement damage are below the filesystem layer; they are hardware-level concerns. Mention only.
- *Adversarial fault injection.* An attacker with kernel privileges who manipulates the filesystem state directly is outside $\mathcal{E}$. This is the same scope choice as FTRFS v1 (Stage 5 in that roadmap).
- *Hardware SEL recovery.* Latched-up hardware is recovered by power cycling; the filesystem operator's response is to map it to $\perp$, nothing more.
- *Emerging memory technologies.* 3D XPoint, ReRAM, MRAM are positioned but not modelled.
- *DDR5 on-die ECC.* Mentioned as a strengthening assumption to be added in v2; v1 models the unfiltered DRAM matrix as the conservative baseline.
- *The fault-injection metric M6.* Reserved for v2, following the FTRFS Phase E campaign currently under development.

c) Methodological limits.: The author is the sole contributor; no peer review beyond the public mailing-list interactions on linux-fsdevel (referenced in [21]) has shaped this text. The proof of the soundness theorem (Section X) is hand-written; it is not machine-checked. v3 will explore mechanization in a proof assistant such as Coq, Isabelle/HOL, or Lean.

d) Threats to validity.: The principal threat is that the perturbation family $\mathcal{E}$ defined in Section II does not faithfully represent the distribution of real SEE events seen by a deployed filesystem. This is precisely why beam-time measurements are needed for v2: to bind the model to ground truth. The soundness theorem is robust under tightening of $\mathcal{E}$ (any sub-family of $\mathcal{E}$ inherits the result) but fragile under loosening (a perturbation that exceeds RS capacity is, by construction, out of scope).

VIII. Conclusion

We have presented the v1 mathematical scaffolding of BEAMFS, a recovery calculus for filesystem resilience under Single-Event Effects. The contribution of this report is not an implementation, nor a measurement campaign: it is the formal apparatus that subsequent versions will instantiate, measure, and prove against.

The calculus is constructed on a multi-layer physical threat model covering DRAM, SRAM and NAND-flash, with explicit positioning of emerging memory technologies. The five invariants (I1–I5) are stated minimally and mapped directly to kernel constructs. The soundness theorem (Theorem IV.1) shows that the recovery operator drives every consistent state, perturbed by any event within the bounded perturbation family, either to a consistent successor or to fail-closed in a finite number of iterations. Silent corruption is excluded by construction.

The empirical content of v1 is necessarily indirect: a re-reading of the FTRFS v1 dataset through the calculus, and a projected recovery surface from canonical cross-section data. The next versions of the report will close this gap. v2 will document the FTRFS Phase E fault-injection campaign and a corresponding M6 bench metric. v3 will explore proof-assistant mechanization of Theorem IV.1 and a verified-compilation chain for the operator. Beam-time at a recognized accelerator facility is the measurement programme this report is intended to support, with the present formal apparatus as scientific dossier.

This is also v1 in another sense: it establishes authorship and fixes the scope. Subsequent versions will refine; the present fixed point is the right place to draw a line and submit it to public review.

IX. Reproducibility

a) Source repository.: https://github.com/roastercode/beamfs, sub-directory `papers/2026-04-beamfs-v1/`.

b) Companion report.: The FTRFS v1 Technical Report [1] is prerequisite reading. Its empirical dataset is reused in Section VI; its threat model is extended in Section II.

```
cd papers/2026-04-beamfs-v1
make figures   # regenerate matplotlib figures from canonical data
make           # build paper.pdf via lualatex + biber + lualatex
```

d) Figure data generation.: The three figures (Figures 1 to 3) are generated from canonical cross-section curves and JEDEC NAND endurance data by Python scripts in `scripts/`:

- `scripts/gen_data.py` produces `data/fig1.dat` (Weibull fits to Petersen-form SEE cross-sections), `data/fig2.dat` (TLC NAND raw BER versus retention age, empirical Cai/Mielke model), and `data/fig3.dat` (RS-capacity-bounded recovery probability over the LET×fluence grid, Poisson statistics).
- `scripts/gen_figures.py` reads these data and produces `figures/fig1.pdf`, `figures/fig2.pdf`, `figures/fig3.pdf` via matplotlib.

Each script is deterministic: given the canonical input parameters documented in its header, it produces a byte-for-byte identical output.

```
md5sum data/*.dat
# values are committed in scripts/gen_data.py headers
# matching values mean: same parameters, same output.
make
# zero error, zero warning expected.
```

X. Appendix A: Soundness Theorem – Full Proof

We restate Theorem IV.1 for self-containment, then give the full proof. The proof is structured as the three-step argument sketched in Section IV: termination, safety, coverage.

a) Restated theorem.: Let Λ be a layout satisfying invariants I1–I3. Let $\mathcal{G}$ be the recovery operator of Definition V.1, satisfying I4–I5. Let $\mathcal{E}$ be a family of physical perturbations such that, for every event $e \in \mathcal{E}$ and every state s, the number of corrupted symbols in any single RS subblock of $e(s)$ does not exceed the correction capacity $t = \lfloor (n-k)/2 \rfloor$ declared by Λ. Then for every initial consistent state $s_0 \vDash$ and every $e \in \mathcal{E}$, there exists $K \leq |\Lambda|$ such that $\mathcal{G}^K(e(s_0)) \vDash$ or $\mathcal{G}^K(e(s_0)) = \bot$.

b) Proof.: Let $s_0 \vDash$ and $e \in \mathcal{E}$. Define $s^{(0)} = e(s_0)$ and $s^{(i+1)} = \mathcal{G}(s^{(i)})$ for $i \geq 0$, terminating the sequence at the first K such that $s^{(K)} \vDash$ or $s^{(K)} = \bot$.

Termination. Define $N(s)$ as in Lemma V.3: the number of regions of Λ on which the syndrome of s is non-zero. By construction $N(s) \in \{0, 1, \ldots, |\Lambda|\}$. Furthermore, $N(s) = 0$ iff $s \vDash$ (Definition III.4, clause 1 and 2 are equivalent to all syndromes being zero; clause 3 follows from I3 once syndromes are zero on dual-layer regions; clause 4 follows from I2 since sentinels are part of the redundancy coverage). By Lemma V.3, for every i such that $s^{(i)} \notin \{\bot\} \cup \{s : s \vDash\}$, we have $N(s^{(i+1)}) \leq N(s^{(i)}) - 1$. The sequence $(N(s^{(i)}))_i$ is therefore strictly decreasing on the non-terminating part, and bounded below by 0; it must reach 0 in at most $|\Lambda|$ steps, or hit $\bot$ earlier. In either case $K \leq |\Lambda|$.

Safety. We show that the sequence cannot terminate in a state $s^{(K)} \notin \{\bot\} \cup \{s : s \vDash\}$. Suppose for contradiction that it does. By the termination clause, the sequence terminates when $s^{(K)} \vDash$ or $s^{(K)} = \bot$, neither of which holds by hypothesis: contradiction. The sequence therefore terminates in one of the two safe states.

We further show that no intermediate state silently masks a corruption. Inspection of Definition V.1 shows that the only ways the operator returns $s' \neq \bot$ on input s are: (a) all syndromes are zero (skip, return s unchanged), or (b) RS correction succeeded *and* CRC re-verification succeeded *and* sentinels are at canonical values. Case (a) preserves consistency by Proposition V.2. Case (b) restores consistency on the affected region by I3 and I2. Any state returned by the operator therefore either is consistent on the affected region or is $\bot$; silent corruption is impossible by construction.

Coverage. The hypothesis on $\mathcal{E}$ asserts that each event e induces at most t symbol errors per RS subblock. The Reed–Solomon code, by its bounded-distance decoding property, decodes correctly any received word within Hamming distance t of a code word, which is exactly the regime in which step 2 of Definition V.1 returns a valid corrected codeword. The CRC re-verification of step 3 is, by I3, a strict tightening: any corrected word that fails CRC is rejected, even if RS accepted it. Therefore the operator, applied iteratively to $e(s_0)$, restores each affected region to its consistent state, until either all regions are consistent ($N = 0$, $s^{(K)} \vDash$) or a region is found whose RS-corrected codeword fails CRC, indicating corruption beyond the design assumption (out of $\mathcal{E}$), in which case $\bot$ is returned.

This concludes the proof.

c) What the proof does not show.: The proof is for a single application chain on a fixed perturbation e. It does not address (a) interleaved events arriving during the recovery loop itself, which would require a concurrent-execution model; (b) probabilistic statements about the distribution of e in $\mathcal{E}$, which would require a measure on the perturbation family; (c) machine-checkability, which would require mechanization in a proof assistant. All three are noted as v3 work in Section VII.

References

[1] A. Desbrières, *FTRFS: Bringing radiation-robust filesystem principles to contemporary Linux*, Technical Report v1, Apr. 2026. doi: 10.5281/zenodo.19824442. [Online]. Available: https://doi.org/10.5281/zenodo.19824442.

[2] H. Chen *et al.*, “Using Crash Hoare Logic for certifying the FSCQ file system,” in *Proc. 25th Symp. Operating Systems Principles (SOSP)*, ACM, 2015. doi: 10.1145/2815400.2815402.

[3] H. Sigurbjarnarson, J. Bornholt, E. Torlak, and X. Wang, “Push-button verification of file systems via crash refinement,” in *12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016.

[4] NASA Goddard Space Flight Center, *Single event effect criticality analysis (SEECA) guideline*, 1996.

[5] European Space Agency, *ECSS-Q-ST-60-15C: Space product assurance – radiation hardness assurance for EEE components*, 2012.

[6] E. Normand, “Single-event effects in avionics,” *IEEE Transactions on Nuclear Science*, vol. 43, no. 2, pp. 461–474, 1996. doi: 10.1109/23.490893.

[7] C. Fuchs, D. Langer, and C. Trinitis, “FTRFS: A fault-tolerant radiation-robust filesystem for space applications,” in *Architecture of Computing Systems (ARCS)*, Porto: Springer, 2015.

[8] W. Heinrich, “Calculation of cosmic ray induced single event upset rates,” *Radiation Effects*, vol. 34, pp. 143–148, 1977.

[9] E. L. Petersen, “Cross section measurements and upset rate calculations,” *IEEE Transactions on Nuclear Science*, vol. 43, no. 6, pp. 2805–2813, 1996. doi: 10.1109/23.556870.

[10] P. E. Dodd and L. W. Massengill, “Basic mechanisms and modeling of single-event upset in digital microelectronics,” *IEEE Transactions on Nuclear Science*, vol. 50, no. 3, pp. 583–602, 2003. doi: 10.1109/TNS.2003.813129.

[11] R. C. Baumann, “Radiation-induced soft errors in advanced semiconductor technologies,” *IEEE Transactions on Device and Materials Reliability*, vol. 5, no. 3, pp. 305–316, 2005. doi: 10.1109/TDMR.2005.853449.

[12] JEDEC Solid State Technology Association, *JESD89A: Measurement and reporting of alpha particle and terrestrial cosmic ray-induced soft errors in semiconductor devices*, Oct. 2006.

[13] NASA, *NASA-HDBK-4002A: Mitigating in-space charging effects – A guideline*, Mar. 2011.

[14] JEDEC Solid State Technology Association, *JESD79-5: DDR5 SDRAM standard*, On-die ECC for DRAM cells, 2020.

[15] U.S. Department of Defense, *MIL-STD-882E: System safety standard practice*, May 2012.

[16] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, “Error characterization, mitigation, and recovery in flash-memory-based solid-state drives,” *Proceedings of the IEEE*, vol. 105, no. 9, pp. 1666–1704, 2017. doi: 10.1109/JPROC.2017.2713127.

[17] N. R. Mielke *et al.*, “Reliability of solid-state drives based on NAND flash memory,” *Proceedings of the IEEE*, vol. 105, no. 9, pp. 1725–1750, 2017. doi: 10.1109/JPROC.2017.2725738.

[18] JEDEC Solid State Technology Association, *JESD218B: Solid-state drive (SSD) requirements and endurance test method*, 2016.

[19] G. Klein *et al.*, “seL4: Formal verification of an OS kernel,” *Proc. 22nd ACM SIGOPS Symp. Operating Systems Principles (SOSP)*, pp. 207–220, 2009. doi: 10.1145/1629575.1629596.

[20] Linux kernel contributors, *lib/reed_solomon: Reed-Solomon error correction library*, Linux kernel source tree, `lib/reed_solomon/`, 2026. [Online]. Available: https://www.kernel.org/.

[21] A. Desbrières, *FTRFS RFC v3 thread on linux-fsdevel*, Public archive, lore.kernel.org, Apr. 2026. [Online]. Available: https://lore.kernel.org/linux-fsdevel/20260414120726.5713-1-aurelien@hackers.camp/T/.